Studio e confronto: il Machine Learning del corso (L9–L13, L18) e la sua applicazione al progetto $B^{\pm} \to K^{\pm}K^{+}K^{-}$

Giacomo Moretti

Ottobre 2026

Come usare questo documento

Per ogni lezione trovi: (A) il concetto spiegato con le formule, (B) che cosa fa il corso (slide ed esercizio), (C) che cosa abbiamo fatto noi, con le configurazioni reali e i numeri, (D) il confronto corso ↔ progetto, (E) dove sta nel report, (F) le domande d'orale con la risposta breve. Le sorgenti sono: slide di *Lecture 9, 10, 11, 12, 13, 18*; il notebook `ML_Nominal.ipynb` e `ml_nominal.py`; i risultati in `results/nominal/*.json`; il report (`ml.tex`, Appendice D e appendici estese). Tutti i numeri sono quelli del report.

0. Il quadro generale

- **Il problema fisico non è un problema di ML**: dobbiamo stimare l'asimmetria raw $A_{\text{raw}} = (N^- - N^+)/(N^- + N^+)$ con un intervallo di incertezza e un test di $A = 0$. Lo fanno il fit frequentista e il fit bayesiano.
- **Il ML ha tre ruoli**, tutti ausiliari:
  1. *classificazione* (supervisionata, con etichette deboli): costruire una selezione signal-like/background-like e verificare se tagliare sul punteggio cambia A_{raw};
  2. *clustering* (non supervisionato, K-means): guardare asimmetrie locali sul piano di Dalitz;
  3. *incertezza* della predizione: confronto tra tre stimatori (spread della Random Forest, MC dropout, deep ensemble).
- **Il risultato**: un taglio > 0.5 sul punteggio del gradient boosting tiene il 92% del segnale e il 36% del fondo e non cambia A_{raw} oltre lo spread statistico atteso (-0.0451 ± 0.0087 bayesiano, -0.0447 ± 0.0086 frequentista, contro -0.0443 e -0.0441); anche il taglio più stretto al massimo della significance improvement (> 0.73, 11 658 candidati) lascia A_{raw} entro 0.9 volte lo spread atteso (-0.0404 ± 0.0097 bayesiano, -0.0401 ± 0.0097 frequentista). Il ML non migliora la precisione perché c'è poco fondo; non può rimuovere il fondo mal identificato sotto il picco.

Lezione	Tema	Che cosa usiamo
L9	statistical learning, bias–varianza, loss come NLL	etichette deboli, split, BCE/Brier, bias–varianza del GBM
L10	regolarizzazione, PCA, K-means	scaler, PCA come test, scan di C per L1/L2, K-means $K = 8$
L11	alberi, bagging, boosting, ROC/AUC	albero, RF, GBM, ROC/AUC, importanze
L12	reti feed-forward	FFNN $20 \to 32 \to 16 \to 1$
L13	addestramento, ottimizzatori, dropout, sbilanciamento	Adam, early stopping, dropout, pesi di classe, calibrazione
L18	incertezza in ML	RF spread, MC dropout, deep ensemble

1 L9 — Statistical learning

A. Concetti

Il modello di fondo.

Si assume che l'uscita sia $Y = f(X) + \varepsilon$: f è la relazione sconosciuta tra le p variabili d'ingresso (feature) e l'uscita (target, label); ε è rumore irriducibile (in fisica: risoluzione del rivelatore, pile-up). Imparare significa stimare $\hat{f}$ dai dati di training $(x_1, y_1), \ldots, (x_N, y_N)$.

I tre archetipi.

- **Regressione**: Y quantitativo (stimare un valore).
- **Classificazione**: Y in un insieme finito e non ordinato (assegnare una classe).
- **Clustering** (non supervisionato): nessun Y, si cercano gruppi di oggetti simili.

Supervised vs unsupervised: nel primo caso si usano esempi etichettati; nel secondo è più difficile dire se si sta facendo bene, perché non c'è un obiettivo noto. I metodi non supervisionati (PCA, clustering) possono preparare i dati per un passo supervisionato.

Che cosa vogliamo da f.

(i) predire casi non visti; (ii) capire quali componenti di X contano; (iii) valutare la qualità di predizioni e inferenze. Il bilancio tra (i) e (ii) è il bilancio tra flessibilità e interpretabilità.

Flessibilità vs interpretabilità.

Un modello molto flessibile segue bene i dati di training ma rischia l'*overfitting*; un modello semplice è leggibile ma può perdere struttura (*underfitting*). Per l'*inferenza* si sceglie interpretabilità, per la *predizione* flessibilità.

Errore di test e decomposizione bias–varianza.

L'errore rilevante è quello su dati *non usati* per costruire il modello. Per l'errore quadratico in un punto x_0:

$$E[(y_0 - \hat{f}(x_0))^2] = \underbrace{\mathrm{Var}(\hat{f}(x_0))}_{\text{varianza}} + \underbrace{[\mathrm{Bias}\,\hat{f}(x_0)]^2}_{\text{bias}^2} + \underbrace{\mathrm{Var}(\varepsilon)}_{\text{irriducibile}} .$$

Il *bias* viene dal semplificare troppo il problema; la *varianza* misura quanto $\hat{f}$ cambierebbe con un altro training set. Aumentando la flessibilità il bias cala e la varianza cresce: l'errore di test ha un minimo a complessità intermedia.

Parametrici vs non parametrici.

I modelli parametrici hanno un numero fisso di parametri e assumono una forma (rigidi, rapidi, rischio di bias alto se la forma è sbagliata, es. regressione lineare). I non parametrici crescono con i dati (flessibili, richiedono più dati, rischio di varianza alta, es. alberi, clustering).

La loss come verosimiglianza.

Se $\Pr(y \mid x) = \Pr(y \mid \theta)$ con $\theta = f[x, \phi]$ e i dati sono i.i.d., il criterio di massima verosimiglianza minimizza la *negative log-likelihood*:

- rumore gaussiano di σ costante: $-\ln L = \sum_i (y_i - f(x_i))^2/(2\sigma^2) + \text{cost.}$ $\Rightarrow$ **MSE** (nel fit ai bin, χ^2);
- uscita binaria di Bernoulli con $p_i = f(x_i)$: $-\ln L = -\sum_i [y_i \ln p_i + (1 - y_i)\ln(1 - p_i)]$ $\Rightarrow$ **binary cross-entropy** (BCE).

Con uscite in $(0, 1)$ la BCE punisce un errore sicuro in modo logaritmico: "esplode" se la predizione è sbagliata e sicura, molto più dell'MSE. È lo stesso principio dei nostri fit estesi: massimizzare la probabilità dei dati osservati.

Rasoio di Occam.

A parità di adattamento si preferisce il modello con meno parametri; la regolarizzazione (L10) è il modo matematico per farlo.

B. Nel corso

Slide con esempi (antispam, cifre di zip code, infarto, LANDSAT, classificazione di collisioni all'LHC). Esercizio: stelle della sequenza principale, polinomi di grado crescente per L in funzione di T (legge di Stefan–Boltzmann, $L \propto T^b$ con b tra 5 e 7); domanda: quando il modello va in overfitting?

C. Nel nostro progetto

- **Formulazione**: classificazione binaria supervisionata, ma con **etichette deboli**: nessun candidato ha la verità segnale/fondo. Si addestra tra due *misture*:
  - signal-like: $|m - \mu| < 2\sigma$ attorno al picco ($\mu = 5284.39$, $\sigma = 18.00$ MeV): **15 130** candidati;
  - background-like: sidebands $[5150, \mu - 4\sigma] \cup [\mu + 4\sigma, 5600]$: **1 940** candidati;
  - la regione 5100–5150 MeV è esclusa dal fondo perché contiene fondo parzialmente ricostruito con cinematica diversa.
- **Perché funziona** (CWoLa, Neyman–Pearson): se le due misture hanno le stesse densità di segnale $s(x)$ e fondo $b(x)$ ma frazioni di segnale diverse $f_1 \neq f_2$, il rapporto delle densità $p_1/p_2 = [f_1 s + (1 - f_1)b]/[f_2 s + (1 - f_2)b]$ è monotono in s/b: il classificatore ottimo tra le misture è, a meno di una trasformazione monotona, quello ottimo tra segnale e fondo. *Condizione*: le feature non devono portare informazione sulla massa.
- **Feature**: 20. Per ognuna delle 3 tracce: `PX`, `PY`, `PZ`, `ProbK`, `ProbPi`, `IPChi2` (18), più `B_FlightDistance` e `B_VertexChi2` (variabili del B non usate nella preselezione; Kolmogorov–Smirnov $p < 10^{-60}$ tra le classi).
- **Split** 60/20/20 stratificato: 10 242 / 3 414 / 3 414 candidati; nel test solo **388** background-like, quindi differenze di AUC sotto ~ 0.01 non si risolvono.
- **Errore di test**: il test non è mai usato per una scelta; il modello di riferimento (gradient boosting) è scelto sull'AUC in cross-validation a 5 fold stratificati.
- **Bias–varianza**: gradient boosting con 0–300 stadi: il gap AUC train–test cresce da 0.018 (20 stadi) a 0.084 (300 stadi); l'AUC di test sale fino a ~ 0.865 da ~ 150 stadi e poi resta piatta. I 200 stadi di default non vanno in overfitting sull'AUC di test.
- **Loss come NLL**: BCE come NLL di Bernoulli; il Brier score $\frac{1}{N}\sum(p_i - y_i)^2$ è l'MSE della probabilità.
- **Parametrico/non parametrico**, **interpretabile/flessibile**: tenere la logistica (0.771) e l'albero singolo (0.824) come baseline leggibili accanto agli ensemble (0.860–0.864).

D. Confronto corso ↔ progetto

Corso	Noi	Perché
etichette vere (MC)	etichette **deboli** da regioni di massa	i dati reali non hanno verità; la validità poggia su CWoLa
regressione e MSE	classificazione e BCE (Brier per valutare)	l'uscita è una probabilità di classe
overfitting con polinomi	overfitting con stadi del GBM	stessa idea: più complessità, gap train–test crescente
accuratezza	AUC; accuratezza sotto la classe maggioritaria (0.886)	con pesi bilanciati si minimizza l'errore *pesato*

E. Dove nel report

Corpo: Sec. ML "Weak Labels and Feature Set" (CWoLa, split, pesi). Appendice ML estesa: "Regularization, Bias-Variance, and Calibration". Appendice D: BCE come NLL, Brier, decomposizione.

F. Domande d'orale

- *Perché classificazione e non regressione?* L'oggetto da stimare è l'appartenenza a una classe (segnale/fondo); l'uscita è una probabilità.
- *Come si fa ML senza etichette vere?* CWoLa: tra due misture con frazioni diverse il classificatore ottimo è monotono in s/b (lemma di Neyman–Pearson). Va verificato che le feature non rivelino la massa (*sculpting*): addestrando senza i nove impulsi dei kaoni l'AUC resta 0.860 e la degradazione del χ^2 del fit non cambia.
- *Cos'è l'overfitting e come lo vedete?* Gap train–test crescente con la complessità (GBM: $0.018 \to 0.084$).
- *Perché BCE e non MSE?* È la NLL della Bernoulli; ha gradiente proporzionale all'errore sulla probabilità anche quando l'uscita è saturata, l'MSE no.

2 L10 — Regolarizzazione e riduzione di dimensione

A. Concetti

Curse of dimensionality.

In molte dimensioni il volume cresce esponenzialmente: per mantenere la stessa densità servono dati esponenzialmente più numerosi; lo spazio è quasi vuoto anche con milioni di eventi; le distanze tra punti diventano indistinguibili (e gli algoritmi basati su distanza soffrono); il rischio di overfitting e il costo computazionale crescono.

Regolarizzazione.

Si modifica la loss: $L_{\text{reg}}(\theta) = L(\theta) + \lambda R(\theta)$, con λ il peso della penalità R (che codifica una proprietà desiderata dei parametri). Per un modello lineare con RSS:

$$\text{Ridge (L2):} \quad \min_{\beta} \ \text{RSS} + \lambda \sum_j \beta_j^2, \qquad \text{Lasso (L1):} \quad \min_{\beta} \ \text{RSS} + \lambda \sum_j |\beta_j|.$$

- **Ridge**: soluzione chiusa $\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$; contrae tutti i coefficienti, ma non ne azzera nessuno. Aumentando λ: varianza $\downarrow$, bias $\uparrow$.
- **Lasso**: porta alcuni coefficienti *esattamente* a zero, quindi fa *selezione di feature*: più interpretabile. Da preferire se bastano poche feature; Ridge se la risposta dipende da molte feature con importanza simile.
- **Formulazione a budget**: $\min \text{RSS}$ con $\sum \beta_j^2 \le s$ (Ridge, vincolo sferico) o $\sum |\beta_j| \le s$ (Lasso, vincolo a rombo con spigoli sugli assi): l'ellisse dei livelli di RSS tocca il rombo di solito su uno spigolo, dove un coefficiente è zero.

Come si sceglie λ: cross-validation.

Suddividere i dati in k fold; per $i = 1, \dots, k$ addestrare su tutte tranne la i-esima e calcolare l'errore sulla i-esima; mediare. Si prende il λ che minimizza l'errore di test stimato. Una sola divisione train/validation dà stime variabili.

Preparazione dei dati.

Garbage in, garbage out: dati inesatti, incompleti, distorti, rumorosi, outlier. Lo **standard scaler** porta ogni feature a media ~ 0 e $\sigma \sim 1$: evita che una feature domini solo perché ha un'unità diversa e aiuta la convergenza. Alternativa per le reti: min–max in $[0, 1]$.

PCA.

Metodo non supervisionato e non parametrico. Date X ($n \times p$, colonne centrate), la prima componente φ_1 massimizza la varianza dei punteggi $z_{i1} = \varphi_1^T x_i$ con $\|\varphi_1\| = 1$ (equivalente: la retta più vicina ai punti in distanza quadratica). La seconda è ortogonale alla prima, e così via. Si risolve con la SVD di X (o l'autodecomposizione di $X^T X$). La *proporzione di varianza spiegata* dalle prime m componenti dice quante tenerne. PCA e Lasso hanno scopi diversi: PCA ruota senza guardare il target, Lasso seleziona guardando il target.

K-means.

K cluster, fissato in anticipo. Si minimizza

$$W = \sum_{\ell=1}^{K} \sum_{i \in C_\ell} \|x_i - \bar{x}_\ell\|^2$$

(inerzia). **Algoritmo di Lloyd**: etichette iniziali casuali; si ripete: (i) calcolo dei centroidi, (ii) riassegnazione di ogni punto al centroide più vicino. Termina sempre (l'inerzia non aumenta),

ma può fermarsi in un minimo *locale* diverso a seconda dell'inizializzazione. Domande da porsi: il clustering è appropriato? quanti cluster? i cluster sono robusti (altri sottocampioni, altri algoritmi)? vanno scalate le variabili? la distanza euclidea cattura la dissimilarità?

B. Nel corso

Esercizio su 20 sensori (5 buoni, 5 distorti, 10 rotti) per vedere quali feature seleziona Lasso; esercizio Higgs con le domande: "tutte le variabili contano allo stesso modo?" e "ha senso Ridge/Lasso con RSS in un problema di classificazione?" (risposta: si usa la logistica con penalità, cioè BCE più penalità).

C. Nel nostro progetto

- **Scaler**: `StandardScaler().fit(X_train)`, poi applicato a validation e test.
- **PCA come test**: sulle 20 feature standardizzate servono **15** componenti per il 90% della varianza ⇒ nessuna riduzione di dimensione applicata (le feature restano interpretabili).
- **Curse of dimensionality**: con 20 feature e $\sim 10^4$ candidati di training non siamo nel regime critico per i metodi basati su distanza.
- **Ridge e Lasso come logistica penalizzata**: scansione di $C = 1/\alpha$ su 30 valori log-spaziati, cross-validata (la logistica è BCE + penalità): AUC ottimo **uguale** per L1 e L2, 0.7686, a $C_{L1} = 0.12$ e $C_{L2} = 9.2$. L1 **non azzera nessun coefficiente**: il modello lineare non soffre di varianza; il suo limite è il confine di decisione lineare.
- **K-means** sul piano di Dalitz $(m^2_{\text{lo}}, m^2_{\text{hi}})$ standardizzato, `n_init=10`:
  - $K = 8$ fissato in anticipo (compromesso tra risoluzione e numero di candidati per cluster, minimo 1 233); $K = 3$ è l'ottimo della silhouette (0.46 contro 0.41 per $K = 8$) ed è aggiunto come confronto;
  - stabilità: indice di Rand aggiustato (ARI) > 0.998 cambiando inizializzazione;
  - significatività locale: $S_k = (A_k - A_{\text{glob}})/\sigma(A_k)$; per $K = 8$ due cluster oltre 3σ, omogeneità $\chi^2 = 46.4/7$ ($p = 7 \cdot 10^{-8}$); il più significativo ha $A = -0.127 \pm 0.017$ ($n = 3601$, $S = -5.1$) a basso m^2_{lo} e m^2_{hi} intermedio; due cluster positivi ad alto m^2_{hi} (+0.020, +0.040); finestra del segnale: $\chi^2 = 62.3/7$.

D. Confronto corso ↔ progetto

Corso	Noi	Nota
Ridge/Lasso con RSS	logistica L1/L2 (BCE)	risposta all'esercizio "ha senso RSS in classificazione?"
PCA per comprimere	PCA per verificare che non serve	15 componenti su 20
K per ispezione	K fissato a priori + silhouette di controllo	silhouette e ARI sono oltre il corso (citati)
K-means sul set di feature	K-means solo su 2 variabili	analisi *locale* dell'asimmetria, non preprocessing

E. Dove nel report

Corpo: "K-means on the Dalitz Plot" e, nel Confronto, "Local Asymmetries". Appendice ML estesa: regolarizzazione. Appendice D: penalità, inerzia, ARI.

F. Domande d'orale

- *Ridge vs Lasso?* L2 contrae tutti i coefficienti, L1 ne azzera alcuni (selezione); geometria: vincolo sferico vs a rombo.
- *Come scegliete λ (o C)?* Cross-validation sul solo training; noi 30 valori log-spaziati, 5 fold.
- *Perché non fate PCA?* Servono 15 componenti su 20: guadagno piccolo, si perderebbe l'interpretabilità delle feature (per esempio `IPChi2`).
- *Il K-means è globalmente ottimo?* No, minimo locale; per questo `n_init=10` e l'ARI tra inizializzazioni.
- *Perché $K = 8$?* Compromesso risoluzione/statistica scelto a priori; $K = 3$ (silhouette) dà lo stesso quadro ($\chi^2 = 15.2/2$, $p = 5 \cdot 10^{-4}$).

3 L11 — Metodi ad albero

A. Concetti

Albero di decisione (CART).

Partizioni binarie ricorsive dello spazio delle feature in rettangoli: ogni nodo interno applica una domanda $x_j \le t$ (sì $\to$ figlio sinistro, no $\to$ destro), ogni foglia ha una predizione. È la generalizzazione di un'analisi a tagli, ma senza scartare un evento che fallisce un criterio. I tagli sono allineati agli assi: le diagonali si approssimano con molti tagli. Ha alta interpretabilità ed è *non lineare*.

Impurità.

L'errore di classificazione 0/1 è discontinuo e degenere (molti tagli danno lo stesso valore). Si usano misure continue, con p_c la frazione della classe c in un nodo:

$$\text{Gini: } G = 1 - \sum_c p_c^2, \qquad \text{entropia: } H = -\sum_c p_c \ln p_c.$$

Lo split sceglie la feature j e la soglia t che massimizzano la riduzione di impurità pesata: $\Delta = G_{\text{padre}} - \frac{n_L}{n} G_L - \frac{n_R}{n} G_R$.

Greedy.

A ogni nodo si scandiscono tutte le feature e tutte le soglie e si prende il taglio migliore localmente, senza lookahead: non si garantisce l'ottimo globale (che richiederebbe di generare tutte le combinazioni).

Quando fermarsi.

Un albero fino a nodi puri è overfitting. Regole di stop: dimensione minima dei figli, profondità massima, numero massimo di nodi, riduzione minima di impurità; oppure cross-validation su una griglia di condizioni; oppure *potatura* a complessità di costo: si minimizza errore $+\alpha\cdot$(numero di nodi), con α scelto per CV.

Alberi di regressione.

Due modifiche: lo split minimizza l'MSE pesato dei figli (equivale a minimizzare la varianza) e la predizione in una foglia è la media dei y del training in quella foglia.

Pro e contro del singolo albero.

Pro: poca preparazione dei dati, variabili continue e categoriche, interpretabilità. Contro: overfitting, sensibilità alle fluttuazioni del training, ottimo globale non garantito, scelta della profondità.

Ensemble: combinare weak learner.

- **Bagging** (bootstrap aggregation): si campionano i dati con reinserimento e si addestra un albero (poco regolarizzato, alta varianza e basso bias) su ogni campione; predizione = voto o media. **Riduce la varianza**. *Random Forest* = bagging + sottoinsieme casuale di feature a ogni split, per decorrelare gli alberi.
- **Boosting**: alberi molto regolarizzati (basso varianza, alto bias) *in sequenza*, ciascuno con più peso sugli esempi sbagliati dai precedenti; media pesata. **Riduce il bias**. Varianti: AdaBoost (pesi sugli esempi), *gradient boosting* ($F_m = F_{m-1} + \nu h_m$, con h_m addestrato sul gradiente negativo della loss, ν learning rate), XGBoost.
- Parametri del boosting: loss (logistica binaria per due classi), learning rate (più piccolo = meglio ma più lento), subsample, numero di alberi (più alberi = rischio overfitting).

ROC e AUC.

Per ogni soglia: TPR = TP/(TP + FN) (sensibilità), FPR = FP/(FP + TN) = 1 − specificità. La curva ROC è TPR contro FPR; l'AUC è l'area ed è la probabilità che un esempio positivo scelto a caso riceva un punteggio maggiore di un negativo scelto a caso (0.5 casuale, 1 perfetto).

Importanza delle feature.

Impurità (Gini) eliminata dagli split su quella feature, sommata sull'ensemble; oppure |coefficiente| del Lasso sulle feature standardizzate. Le due misure vanno confrontate (esercizio Higgs).

B. Nel corso

Esercizio "Circular correlation with trees": un modello lineare non separa due classi circolari, un albero sì. Esercizio Higgs: BDT, importanza delle feature, confronto con i coefficienti Lasso (concordano? perché le variabili di alto livello discriminano meglio?), accuratezza di logistica penalizzata contro BDT.

C. Nel nostro progetto

Tutti i modelli usano le stesse feature scalate e pesi di classe bilanciati ($w_c = N/(2N_c)$: 4.40 per background-like, 0.56 per signal-like).

Modello	Configurazione	CV AUC	Test AUC	Brier
Logistica	L2 (default), pesi bilanciati	0.769 ± 0.016	0.771 ± 0.013	0.197
Logistica L1	`liblinear`	–	0.771 ± 0.013	0.198
Albero	depth 4, `min_samples_leaf`=20, Gini	0.816 ± 0.011	0.824 ± 0.013	0.133
Random Forest	300 alberi, depth 8, `min_samples_leaf`=20	0.863 ± 0.017	0.860 ± 0.012	0.122
Gradient Boosting	200 stadi, depth 3, lr 0.05, sample weights bilanciati	$\mathbf{0.866 \pm 0.016}$	0.864 ± 0.012	0.116
FFNN (L12)	$32 \to 16$, ReLU, dropout 0.2	–	0.857 ± 0.012	0.134

- **Riferimento**: il GBM, scelto sull'AUC in cross-validation (non sul test). RF e FFNN sono indistinguibili entro l'incertezza del test (0.012; differenze reciproche ≤ 0.007); i tre sono ~ 0.09 sopra la logistica e ~ 0.04 sopra l'albero singolo.
- **Albero singolo**: profondità fissata a priori come baseline interpretabile, non potato; il guadagno di AUC del bagging/boosting (0.04) è il costo in varianza dell'albero singolo.
- **Bagging contro boosting**: RF (bagging, taglia la varianza) e GBM (boosting, taglia il bias) danno lo stesso AUC entro l'incertezza.
- **ROC/AUC**: incertezza di test per bootstrap (2 000 ricampionamenti); punto di lavoro score > 0.5: 88% del signal-like e 28% del background-like sul test; 92% e 36% sul campione completo. La figura della ROC ha un secondo pannello con la significance improvement $\varepsilon_S/\sqrt{\varepsilon_B}$ (extra, non nel corso): massimo 1.84 a score 0.73 per il GBM, 1.67 a 0.5; non è il taglio ottimo della misura, perché qui il segnale domina e la precisione segue $S/\sqrt{S+B}$.
- **Importanza**: Gini della RF contro |coefficiente| Lasso: Spearman $\rho = 0.42$ ($p = 0.06$, 20 feature; la correlazione di Spearman è un extra, non nel corso: confronta due classifiche senza assumere la scala; concordano solo in parte). Le più utili: `IPChi2` dei tre adroni e le due variabili del B (`B_FlightDistance`, `B_VertexChi2`) non usate nella preselezione, analogo del risultato "le variabili di alto livello discriminano meglio" dell'esercizio.

- **Ricerca di iperparametri** (oltre l'esercizio): RF 120 configurazioni, GBM 100 (griglia $5 \times 5 \times 4$), FFNN 168; la CV è ripetuta. Miglioramento dell'AUC di test ≤ 0.005, sotto l'incertezza (0.012): i default sono vicini all'ottimo; il massimo in CV è ottimistico (*winner's curse*: 0.871 contro 0.866 e 0.868 contro 0.865).
- **Uso nella fisica**: il punteggio del GBM si applica a *tutti* i candidati della finestra, non solo a quelli etichettati; tagli da 0.1 a 0.9; ricalcolo di A_{raw} con conteggio e con i due fit. Lo spread atteso tra un campione annidato e il completo è $\sigma_B = \sqrt{\sigma^2_{\text{sub}} - \sigma^2_{\text{full}}}$ (Barlow); lo shift resta entro $1.4\,\sigma_B$ su tutta la scansione: nessun bias rivelabile.
- **Sculpting (cambia la forma della massa?)**: sì, il χ^2 del fit passa da 164/88 a 253/88 (bayesiano) dopo il taglio. Due ipotesi: il classificatore ricostruisce la massa dagli impulsi, oppure la selezione cambia la composizione del fondo. Il test: addestrare senza i nove componenti di impulso (AUC 0.860) dà la stessa degradazione ($\chi^2 = 533/388$) $\Rightarrow$ è la composizione del fondo; l'efficienza del taglio vs massa lo conferma (0.89–0.90 al picco, ≈ 0.70 sotto 5150 MeV, 0.06–0.13 sopra 5375 MeV).

D. Confronto corso ↔ progetto

Corso	Noi	Nota
BDT su Higgs con etichette MC	GBM con etichette deboli	stessa tecnica, provenienza diversa delle etichette
alberi di regressione	non usati	il problema è di classificazione
potatura a complessità di costo	non usata	profondità fissa e CV sugli iperparametri
AdaBoost/XGBoost	GBM di scikit-learn (Friedman)	stessa famiglia; nessuna differenza attesa sotto 0.012
subsample del boosting	non usato (default 1.0)	–
confronto con Lasso	fatto, Spearman 0.42 (extra)	stessa conclusione qualitativa
significance improvement $\varepsilon_S/\sqrt{\varepsilon_B}$	ROC + SIC, taglio al massimo (0.73)	extra: non nel corso; ha senso per un segnale piccolo, qui test di stabilità

E. Dove nel report

Corpo: "Model Comparison" (tabella dei modelli, ROC) e "Stability of A_{raw} under ML Selection" (tabella dei tre metodi, soglia). Appendice ML estesa: importanze, iperparametri (tabelle di ricerca), bias–varianza, calibrazione.

F. Domande d'orale

- *Bagging vs boosting?* Bagging: alberi profondi su bootstrap, media (taglia la varianza). Boosting: alberi superficiali in sequenza che correggono gli errori (taglia il bias).
- *Perché Gini/entropia e non l'errore 0/1?* L'errore 0/1 è discontinuo e degenere; Gini ed entropia sono continue e premiano la purezza.
- *Come evitate l'overfitting dell'albero?* Profondità massima e minimo di candidati per foglia; per gli ensemble stadi e learning rate; valutazione su test mai toccato.
- *Cosa misura l'AUC?* La probabilità che un signal-like riceva un punteggio maggiore di un background-like (0.864 per il GBM).

- *Perché il GBM è il riferimento?* Migliore AUC in CV (0.866); le altre sono entro l'incertezza; la scelta non usa il test.
- *Il taglio cambia la forma della massa: è sculpting?* No: stessa degradazione senza gli impulsi; è la composizione del fondo (il classificatore tiene i B genuini, anche parzialmente ricostruiti, e toglie il combinatorio).

4 L12 — Reti neurali feed-forward

A. Concetti

Perché non basta un modello lineare.

Un modello lineare stima un peso per ogni feature, indipendentemente: ma spesso le feature di basso livello (pixel, impulsi) non hanno significato da sole, conta la relazione tra esse. Una rete combina feature in modo non lineare, in strati.

Che cos'è.

Una composizione di funzioni (strati) collegate a catena; ogni neurone calcola la somma pesata degli ingressi, aggiunge un bias e applica una *funzione di attivazione*:

$$a^{(\ell)} = \varphi(W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}), \qquad a^{(0)} = x.$$

Feed-forward: informazione da sinistra a destra, nessun ciclo. Usi: regressione e classificazione (a due o più classi).

Perché serve la non linearità.

Senza attivazioni non lineari una pila di strati lineari è ancora una trasformazione lineare. Le attivazioni non lineari permettono di apprendere pattern complessi.

Attivazioni degli strati nascosti.

ReLU $\varphi(z) = \max(0, z)$: la più usata; consente dinamica non lineare, converge velocemente perché non satura per $z > 0$, non soffre di *vanishing gradient* per $z > 0$. La **sigmoide** $\sigma(z) = 1/(1 + e^{-z})$ non andrebbe usata in genere per gli strati densi/convoluzionali: il gradiente svanisce lontano da $z = 0$ e l'uscita non è centrata in zero (zig-zag in SGD).

Attivazioni dello strato di uscita.

Identità: regressione. *Sigmoide*: classificazione binaria con un neurone (uscita $= P(y = 1 \mid x)$). *Softmax*: $\mathbb{R}^n \to [0, 1]^n$ con $\sum y_i = 1$, classi mutuamente esclusive.

Teorema di approssimazione universale.

Una rete con un solo strato nascosto e un numero finito di neuroni con attivazione non lineare può approssimare funzioni continue su compatti. *Importante*: il teorema non dice nulla su quanto sia semplice *apprendere* i parametri; tutta la pratica del DNN è trovare tecniche efficienti per farlo.

Perché andare in profondità.

Un solo strato è sufficiente in linea di principio, ma le architetture profonde rappresentano più efficientemente classi di funzioni (meno parametri), riusano sotto-feature per più compiti, e la sovraparametrizzazione sembra rendere la loss più liscia (convergenza più facile).

Addestramento (anticipazione di L13).

Pesi e bias sono inizializzati a caso e aggiustati con discesa del gradiente e backpropagation su una loss differenziabile; gli *iperparametri* (numero di strati, neuroni, dropout, learning rate) con prove e griglie. **Curve di apprendimento**: l'errore di training cala e raggiunge un plateau che dipende da dimensione del training, architettura e pesi iniziali; se l'errore di validation risale c'è overfitting.

Altre architetture.

Le FFNN sono modelli universali ma troppo flessibili; le architetture moderne introducono priori (CNN per immagini, ecc.).

B. Nel corso

TensorFlow Playground (XOR e spirale: rete minima, bias/varianza); esercizi con Keras sul dataset Higgs (confronto FFNN vs alberi), classificazione multiclasse di jet, regressione su potenza solare. Si usa Keras (modello sequenziale).

C. Nel nostro progetto

- **Architettura**: 20 ingressi $\to$ 32 (ReLU, dropout 0.2) $\to$ 16 (ReLU, dropout 0.2) $\to$ 1 (sigmoide). Parametri: $20 \cdot 32 + 32 = 672$, $32 \cdot 16 + 16 = 528$, $16 + 1 = 17$: totale **1 217**.
- **Loss e metrica**: BCE; monitoraggio dell'AUC su validation; input scalati sul training.
- **Esito**: AUC di test 0.857 ± 0.012, indistinguibile da RF e GBM; spread su 10 inizializzazioni 0.854 ± 0.005.
- **Ricerca dell'architettura**: 168 configurazioni = 7 architetture $\times$ 4 dropout $\times$ 6 learning rate, CV 5×2; il migliore guadagna 0.004 di AUC di test, paragonabile alla dispersione sulle inizializzazioni (0.005): i default sono sufficienti.
- **Perché la rete non batte gli alberi**: dati tabulari eterogenei, $\sim 10^4$ esempi di training e poche feature: regime in cui gli ensemble di alberi sono tipicamente altrettanto buoni; una rete più grande non troverebbe informazione in più.

D. Confronto corso $\leftrightarrow$ progetto

Corso	Noi	Nota
FFNN su Higgs (confronto con BDT)	FFNN sui nostri dati, confronto con GBM	stessa conclusione: alberi e rete alla pari
sigmoide in uscita per 2 classi	sigmoide, BCE	uscita = probabilità (poi ricalibrata)
curve di apprendimento	early stopping sul val. AUC	–
regressione, multiclasse	non usate	il nostro è binario

E. Dove nel report

Corpo: "Model Comparison" (sesto modello della tabella). Appendice ML estesa: tabella della ricerca FFNN (168 configurazioni).

F. Domande d'orale

- *Perché una rete e non solo la logistica?* Per catturare interazioni non lineari tra feature; la logistica ha limite nel confine lineare (0.771 vs 0.857).
- *Cosa dice il teorema di approssimazione universale?* Esiste una rete con uno strato che approssima una funzione continua su un compatto; non garantisce che si riesca ad addestrarla.
- *Perché ReLU e non sigmoide nei nascosti?* Non satura per $z > 0$, evita il vanishing gradient e converge più rapidamente.
- *Perché sigmoide in uscita?* Un solo neurone, classificazione binaria: uscita in $(0, 1)$ interpretabile come probabilità.

5 L13 — Addestrare una rete

A. Concetti

Discesa del gradiente (GD).

Si parte da $w^{(0)}$; a ogni iterazione si calcola il gradiente della loss $\nabla_w \ell(w^{(t)})$ (direzione di massima crescita) e si fa un piccolo passo opposto:

$$w^{(t+1)} = w^{(t)} - \eta \, \nabla_w \ell\big(w^{(t)}\big),$$

con η il learning rate. Limite: richiede un passaggio sull'*intero* training a ogni passo.

SGD e mini-batch.

Si stima il gradiente su un solo esempio (SGD) o su un mini-batch (es. 8–256 esempi): sfrutta le operazioni vettoriali, riduce la volatilità dei passi (ma il rumore è anche utile), parallelizzabile. Non serve controllare l'errore di validation a ogni iterazione.

Backpropagation.

Regola della catena per attribuire a ogni peso la "colpa" dell'errore. Con $z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$ e $\delta^{(\ell)} = \partial \ell / \partial z^{(\ell)}$:

$$\delta^{(\ell)} = \big(W^{(\ell+1)\,T} \delta^{(\ell+1)}\big) \odot \varphi'(z^{(\ell)}), \qquad \frac{\partial \ell}{\partial W^{(\ell)}} = \delta^{(\ell)} \, a^{(\ell-1)\,T}.$$

Si calcola in avanti l'uscita e all'indietro i δ.

Problemi di addestramento.

- **Vanishing gradient**: con attivazioni saturanti i prodotti di derivate molto piccole cancellano il gradiente degli strati iniziali.
- **Batch normalization**: stabilizza le attivazioni degli strati interni (riduce l'*internal covariate shift*) e accelera l'apprendimento.
- **Loss non convessa**: sensibilità all'inizializzazione, minimi locali, punti di sella (più comuni in alta dimensione); in pratica i minimi *piatti* generalizzano meglio di quelli stretti.
- **Learning rate annealing**: η decresce durante il training.

Ottimizzatori.

- **Momentum**: $v \leftarrow \beta v + g$, $w \leftarrow w - \eta v$ (una "palla con massa" che smorza le oscillazioni nelle direzioni di curvatura alta e accelera in quelle dolci).
- **Learning rate adattivi**: AdaGrad (normalizza per la somma dei quadrati dei gradienti passati), RMSProp/AdaDelta (media mobile con decadimento $\gamma \approx 0.9$).
- **Adam**: stima media e varianza non centrata dei gradienti, $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$, correzione del bias $\hat{m}_t, \hat{v}_t$, aggiornamento $w \leftarrow w - \eta \, \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$; tipico $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Regolarizzazione delle reti.

- **Dropout**: durante il training si spegne a caso una frazione p dei neuroni: impedisce alla rete di dipendere da insiemi particolari di feature (e, a inferenza, è il fondamento dell'MC dropout di L18).
- **Early stopping**: si interrompe quando la loss di validation raggiunge un plateau o risale.

- **Noise injection** e **data augmentation**: trasformazioni che preservano la fisica per rendere più ricco il training.

Dati sbilanciati.

Un modello addestrato su classi sbilanciate va male sulla minoritaria.

- *Oversampling* della minoritaria (rischio di overfitting); *undersampling* della maggioritaria (si perde informazione); **SMOTE** (punti sintetici per interpolazione tra vicini della minoritaria).
- **Pesi di classe**: pesare la loss. Il solo downsampling mostra al modello un mondo artificialmente più bilanciato di quello reale (bias di predizione): la ricetta del corso è *downsample + upweight*: omettere esempi maggioritari e dare più peso alla loss sugli esempi mantenuti; il modello conosce sia il legame feature–label sia la distribuzione reale, e converge più in fretta.

B. Nel corso

Esercizio "implementazione a mano" di una FFNN (uno strato nascosto da 50 nodi) sul dataset dei vini bianchi (11 feature, regressione sulla qualità), confronto con Keras; notebook MSE vs BCE.

C. Nel nostro progetto

- **Ottimizzatore**: Adam, learning rate 10^{-3}, mini-batch di 256, fino a 100 epoche.
- **Early stopping**: sul `val_auc`, `patience`=8, `restore_best_weights=True`.
- **Dropout**: 0.2 negli strati nascosti (valore migliorabile nella ricerca di architettura: dropout variato tra 4 valori).
- **Classi sbilanciate** ($1940/15\,130 = 0.128$): **pesi di classe bilanciati** $w_c = N/(2N_c)$ in tutti i modelli (`class_weight`, e `sample_weight` per il gradient boosting), così le due classi contribuiscono ugualmente alla loss. Il corso suggerisce anche downsample + upweight; nessun esempio viene scartato nella nostra scelta.
- **Conseguenza sulle probabilità**: con i pesi, il modello stima non p ma $\tilde{p} = w_s p/(w_s p + w_b(1-p))$: un mondo "ribilanciato". Con $w_b > w_s$ i punteggi grezzi *sottostimano* la probabilità di signal-like: a punteggio 0.25 la frazione osservata è 0.58.
- **Calibrazione**: regressione isotonica (`CalibratedClassifierCV`, 5 fold) segue la diagonale e dimezza il Brier ($0.122 \to 0.062$); il ranking, e quindi AUC e tagli, non cambia perché la trasformazione è monotona.
- **Riproducibilità**: seed fissi, operazioni deterministiche.

D. Confronto corso ↔ progetto

Corso	Noi	Nota
SGD, mini-batch, momentum, Adam	Adam, batch 256	ottimizzatore adattivo standard
dropout, early stopping	entrambi	validazione sull'AUC
batch norm, annealing, augmentation	non usati	rete piccola, training breve, dati tabulari
SMOTE, over/undersampling	non usati	pesi di classe
downsample + upweight	pesi di classe sul campione intero	stessa idea (correggere la loss) senza scartare esempi

E. Dove nel report

Corpo: “Weak Labels and Feature Set” (pesi di classe) e “Model Comparison” (ottimizzatore, early stopping). Appendice ML estesa: calibrazione (formula di $\tilde{p}$, isotonica, Brier). Appendice D: perdita pesata e ricalibrazione.

F. Domande d’orale

- *Perché Adam?* Combina momentum e learning rate adattivo per parametro; è robusto e richiede poca messa a punto.
- *Come gestite lo sbilanciamento?* Pesi di classe bilanciati; si perde la calibrazione (si stima $\tilde{p}$), che si ripristina con la regressione isotonica senza cambiare ranking.
- *Cos’è l’early stopping e che cosa regolarizza?* Ferma l’addestramento quando la validation smette di migliorare; limita la complessità effettiva della rete.
- *Cosa fa il dropout?* Spegne neuroni a caso: la rete non può contare su singole feature; a inferenza lasciato attivo dà un ensemble (MC dropout).

6 L18 — Incertezza nelle reti

A. Concetti

L'idea.

Un modello dovrebbe restituire una predizione *e* la sua incertezza (un intervallo, una distribuzione). Uncertainty quantification (UQ): sistemi che non fanno solo predizioni ma danno misure di confidenza. Utile per decidere se fidarsi di un'uscita, e per l'*open set recognition* (una softmax classifica comunque un cavallo come gatto o cane).

Aleatoria ed epistemica.

- **Aleatoria**: rumore intrinseco nelle osservazioni (rumore di sensore, di etichetta, sovrapposizione delle classi). *Omoschedastica* (costante) o *eteroschedastica* (dipende dall'input). Non si riduce con più dati, ma con più feature descrittive.
- **Epistemica**: ignoranza del modello sui parametri, per dati di training insufficienti. Si riduce con più dati e svanisce nel limite di dati infiniti (modello identificabile).

Metodi.

- **Bagging** (approccio frequentista): M reti identiche addestrate su M campioni bootstrap (N su N con reinserimento); la predizione è la media delle M uscite, l'incertezza epistemica la loro varianza: si "simula la variabilità della popolazione" con il solo campione disponibile.
- **Deep ensemble**: M istanze della stessa architettura con inizializzazioni diverse, addestrate sullo stesso training; media e deviazione standard.
- **Regressione puntuale con varianza**: testa doppia che predice $\mu(x)$ e $\sigma^2(x)$, loss NLL gaussiana: cattura l'aleatoria ma non l'epistemica (non segnala campioni fuori distribuzione). Variante: *quantile regression*.
- **MC dropout** (Gal e Ghahramani): lasciare il dropout *attivo* a inferenza e ripetere T volte: ogni passaggio è un campione dalla "posterior" dei pesi; la varianza dei passaggi stima l'ignoranza del modello. È equivalente a un'approssimazione variazionale.
- **Reti bayesiane** (Bayes by backprop): una distribuzione su ogni peso, parametri (μ_i, ρ_i); loss = ELBO (verosimiglianza + KL dal prior): con N crescente la verosimiglianza domina e la posterior diventa guidata dai dati; costo: $2\times$ i parametri, addestramento più sensibile ai prior. Il VAE usa lo stesso motore (trucco della riparametrizzazione e ELBO) ma per l'incertezza dello spazio latente.

B. Nel corso

Notebook di Bayes by backprop implementato a mano (regressione); discussione della connessione VAE – MC dropout – BNN.

C. Nel nostro progetto

- **Tre stimatori** confrontati, ognuno con M predizioni $p_m(x)$ per candidato e $\sigma(x) = \left[\frac{1}{M}\sum_m (p_m - \bar{p})^2\right]^{1/2}$:
  - **Random Forest**: $M = 300$ alberi (bagging: ognuno su un bootstrap): incertezza media **0.178**;

  - **MC dropout** sulla FFNN: $M = 300$ passaggi con dropout attivo: **0.095** (0.099 ± 0.006 su 10 inizializzazioni);
  - **Deep ensemble**: $M = 10$ FFNN con inizializzazioni diverse: **0.050**.

- **Perché differiscono (fattore ~ 3.5)**: misurano fonti diverse. Il deep ensemble varia solo l'inizializzazione (stessi dati); MC dropout perturba anche la struttura; gli alberi della RF differiscono per dati (bootstrap) e feature, e ogni albero limitato in profondità è uno stimatore povero e discreto di p. Correlazioni di Spearman tra stimatori 0.41–0.57: correlati, non intercambiabili.
- **Aleatoria vs epistemica** sulla RF: lo spread è massimo per punteggio ≈ 0.5 (le classi si sovrappongono e gli alberi discordano: componente aleatoria) e cala con la dimensione del training, da 0.220 (un quarto) a 0.204 (metà) a 0.179 (completo), mentre l'AUC di test resta a 0.860–0.862: componente epistemica. Ai tre stimatori l'incertezza media è entro il 10% per signal-like e background-like.
- **Limite onesto**: queste incertezze per candidato sono un *diagnostico* del classificatore: non vengono propagate all'incertezza di A_{raw}, che nasce dal fit e dalla sistematica.

D. Confronto corso ↔ progetto

Corso	Noi	Nota
bagging (frequentista)	RF: spread degli alberi	stesso schema, ma alberi e non reti
deep ensemble	10 FFNN con seed diversi	uguale al corso
MC dropout	300 passaggi con `training=True`	uguale al corso
regressione con μ, σ^2, quantile	non usati	classificazione: l'uscita è già una probabilità
Bayes by backprop, ELBO	non usati	implementazione dedicata, MC dropout dà un ensemble simile a costo nullo
VAE	non usato	fuori scopo

E. Dove nel report

Appendice ML estesa: "Uncertainty Quantification" (figura con le tre distribuzioni e lo spread vs dimensione del training). Appendice D: aleatoria/epistemica, varianza dell'ensemble, MC dropout.

F. Domande d'orale

- *Differenza tra incertezza aleatoria ed epistemica?* Aleatoria: sovrapposizione intrinseca delle classi, non cala con più dati; epistemica: ignoranza del modello, cala con più dati.
- *Come l'avete misurata?* Spread di tre ensemble; la parte epistemica si vede nel calo dello spread RF con il training ($0.220 \to 0.179$), l'aleatoria nel picco a punteggio ≈ 0.5.
- *Perché MC dropout stima la posterior?* Tenere il dropout attivo equivale a campionare reti diverse: approssimazione variazionale della posterior dei pesi.
- *Perché i tre valori sono diversi?* Variano cose diverse (dati, struttura, inizializzazione): non è un errore, è una differenza di definizione.
- *Questa incertezza entra in A_{raw}?* No: A_{raw} viene dal fit; le incertezze ML sono un diagnostico del classificatore.

7 Le differenze tra i modelli utilizzati

In una riga ciascuno

Modello	Idea	AUC di test
Logistica (L2/L1)	combinazione *lineare* delle feature, passata in una sigmoide	0.771
Albero	sequenza di tagli sì/no su una feature	0.824
Random Forest	tanti alberi profondi indipendenti, media	0.860
Gradient boosting	alberi piccoli in fila, ognuno corregge i precedenti	0.864
FFNN	strati di neuroni con attivazioni non lineari	0.857
K-means	raggruppa i punti per vicinanza, senza etichette	– (non classifica)

L'incertezza di test dell'AUC è 0.012: RF, GBM e FFNN sono indistinguibili tra loro; la logistica e l'albero sono nettamente sotto.

Come ragionano

Logistica (Ridge/Lasso).

Calcola $z = \beta_0 + \sum_j \beta_j x_j$ e $p = \sigma(z)$: il confine di decisione è un *iperpiano*, lineare. Ridge (L2) riduce tutti i coefficienti, Lasso (L1) ne azzera alcuni. *Pro*: veloce, interpretabile (ogni β_j dice quanto pesa una feature), bassa varianza. *Contro*: non cattura interazioni né confini curvi. Da noi L1 non azzera nessun coefficiente e Ridge e Lasso danno lo stesso AUC: il problema non è la varianza ma il fatto che il confine non è lineare.

Albero.

Taglia lo spazio in rettangoli (tagli allineati agli assi), una predizione per foglia; non lineare. *Pro*: leggibile, nessuno scaler. *Contro*: **alta varianza** (cambia molto con un altro campione) e tendenza all'overfitting. Da noi: profondità 4, `min_samples_leaf`= 20, usato come baseline interpretabile.

Random Forest (bagging).

Molti alberi profondi, ognuno su un campione bootstrap e con un sottoinsieme casuale di feature a ogni split; si media. Riduce la **varianza**: gli errori indipendenti dei singoli alberi si compensano. *Pro*: robusta, poche regolazioni, quasi non va in overfitting. *Contro*: meno interpretabile; ogni albero è un cattivo stimatore di p.

Gradient boosting.

Alberi *superficiali* in sequenza: ognuno impara il gradiente negativo della loss (gli errori residui) e si somma con un peso piccolo ν: $F_m = F_{m-1} + \nu h_m$. Riduce il **bias**. *Pro*: di solito il più accurato su dati tabulari. *Contro*: sequenziale, più sensibile a rumore e iperparametri, overfitting se troppi stadi. Da noi: 200 stadi, profondità 3, $\nu = 0.05$; è il riferimento perché ha il miglior AUC in cross-validation (0.866).

FFNN.

Strati di neuroni: $a^{(\ell)} = \varphi(W^{(\ell)} a^{(\ell-1)} + b^{(\ell)})$, ReLU nascosta e sigmoide in uscita; si addestra con Adam per backpropagation. *Pro*: approssimatore universale, molto flessibile. *Contro*: richiede scaler, più iperparametri (architettura, dropout, learning rate), meno stabile (dipende dall'inizializzazione). Da noi: $20 \to 32 \to 16 \to 1$, ReLU, dropout 0.2, 1 217 parametri, spread sull'inizializzazione 0.854 ± 0.005.

K-means.

Non è confrontabile con gli altri perché non ha etichette: partiziona il piano di Dalitz in K gruppi minimizzando la distanza dai centroidi.

Confronto sinottico

	Logistica	Albero	Random Forest	Gradient boosting	FFNN
Confine	lineare	rettangoli	rettangoli (medio)	rettangoli (somma)	curvo
Bias	alto	medio	basso	molto basso	basso
Varianza	bassa	**alta**	bassa (bagging)	media	media
Interpretabilità	**alta**	alta	media	bassa	bassa
Scaler	sì	no	no	no	sì
Pesi di classe	`class_weight`	`class_weight`	`class_weight`	`sample_weight`	`class_weight`
Parametri chiave	C (penalità)	profondità, foglia minima	n. alberi, profondità	stadi, ν, profondità	strati, dropout, lr

Perché i risultati vengono così

- $0.771 \to 0.824$: dal confine lineare a uno non lineare l'AUC sale di ~ 0.05; il segnale sta nelle interazioni tra feature (per esempio `IPChi2`, `B_FlightDistance`).
- $0.824 \to 0.860$: la media di tanti alberi toglie la varianza dell'albero singolo (~ 0.04).
- 0.860**,** 0.864**,** 0.857: differenze sotto l'incertezza di test (0.012): su dati tabulari con $\sim 10^4$ esempi nessuna famiglia ha un vantaggio netto.

Come spiegarlo all'orale in tre frasi

1. "Ho confrontato modelli di complessità crescente: lineare, albero singolo, ensemble di alberi e rete."
2. "Il salto grande è dal lineare al non lineare e dall'albero singolo agli ensemble; tra RF, GBM e FFNN le differenze sono dentro l'incertezza di test."
3. "Ho scelto il GBM in base all'AUC in cross-validation, senza usare il test, e ho tenuto logistica e albero come baseline interpretabili."

8 Riepilogo trasversale

Che cosa NON abbiamo fatto (e perché)

Argomento	Motivo
alberi di regressione	problema di classificazione
potatura a complessità di costo	albero singolo solo baseline; profondità a priori
AdaBoost, XGBoost	GBM sufficiente; differenze sotto l'incertezza del test
PCA come riduzione	servono 15/20 componenti
selezione di feature con Lasso	L1 non azzera nessun coefficiente
batch normalization, annealing, augmentation	rete piccola, dati tabulari
SMOTE, over/undersampling	pesi di classe + ricalibrazione
regressione con varianza, quantile, Bayes by backprop, VAE	uscita già probabilistica; MC dropout e ensemble bastano
CNN, RNN, GNN, autoencoder (L14–17), unfolding (L19), OmniFold (L20)	non applicabili; l'unfolding compare come limite ("A_{raw} è quella piegata")

Oltre il programma (richiede riferimento o ragione)

CWoLa (Metodiev, Nachman, Thaler), calibrazione isotonica (Zadrozny–Elkan), Brier score, silhouette (Rousseeuw), ARI (Hubert–Arabie), σ_B per campioni annidati (Barlow), test di permutazione sul Dalitz, studio "high-background" con identificazione più larga.

I numeri da ricordare

- Campione: 19 731 candidati nella finestra $[5100, 5600]$ MeV; etichette deboli 15 130/1 940; 20 feature; split 10 242/3 414/3 414; 388 background-like nel test.
- AUC di test: 0.771 (logistica), 0.824 (albero), 0.860 (RF), 0.864 (GBM), 0.857 (FFNN); incertezza 0.012.
- GBM: gap train–test $0.018 \to 0.084$ ($20 \to 300$ stadi); plateau ~ 0.865 da ~ 150 stadi.
- PCA: 15/20 componenti per il 90%; L1/L2: 0.7686; nessun coefficiente azzerato.
- Soglia 0.5: 88%/28% (test), 92%/36% (campione); A_{raw}: -0.0451 ± 0.0087 (bayes), -0.0447 ± 0.0086 (freq), -0.0446 ± 0.0080 (conteggio), contro -0.0443, -0.0441, -0.0432.
- K-means $K = 8$: $\chi^2 = 46.4/7$; cluster più significativo $A = -0.127 \pm 0.017$ ($S = -5.1$); $K = 3$: silhouette 0.46.
- Incertezza media: RF 0.178; MC dropout 0.095; deep ensemble 0.050.

Una frase per ogni ruolo (per l'orale)

- **Classificazione**: "Usiamo un classificatore con etichette deboli per costruire una selezione e verificare che la misura di A_{raw} non ne dipenda; non predice la carica."
- **Clustering**: "Il K-means dà una partizione adattiva del piano di Dalitz, confrontata con la griglia fissa 8×8: stesso quadro."
- **Incertezza**: "Tre stimatori danno scale diverse perché misurano fonti diverse; la parte epistemica cala con i dati."

- **Il limite**: “Il ML non sostituisce i fit: è una verifica di robustezza e un test di sculpting, e non può togliere il fondo mal identificato sotto il picco.”